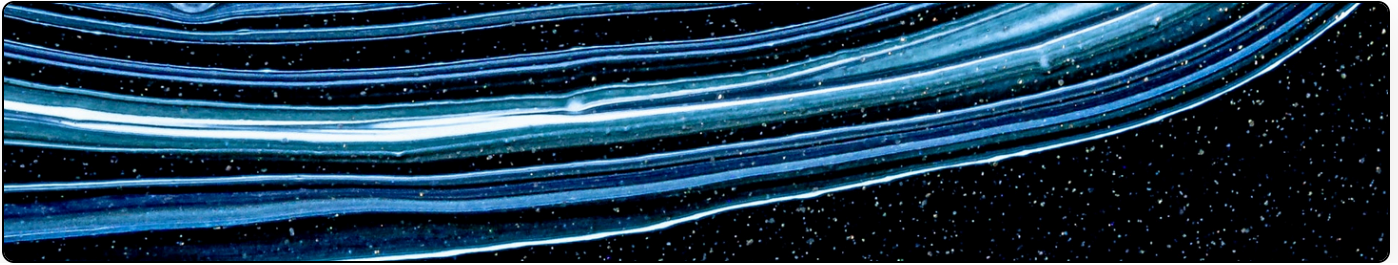


On application observability in serverless cloud contexts

Cite as: Martin Paul Eve, "On application observability in serverless cloud contexts", *eve.gd*:

Martin Paul Eve, March 25, 2023, <https://doi.org/10.59348/sm61j-c6q60>.



I have been thinking, this week, about the observability of AWS Lambda functions in API Gateway contexts. The major challenge is that Prometheus metrics pose a problem as they are pull-only (via a scraping endpoint). Prometheus metrics are stored in a temporary disk cache and then pulled off-site by Grafana etc. But this makes them difficult to collect in the context of ephemeral services, such as Lambda functions, where you can't guarantee a persisted endpoint with that data store.

We had a series of technical goals for observability:

- We our projects to be observable with useful instrumentation.
- We currently have a Grafana dashboard that can ingest Prometheus metrics, CloudWatch metrics/logs, and a range of other sources such as Loki.
- Prometheus metrics pose a challenge as they are pull-only (via a scraping endpoint). This makes them difficult to collect for ephemeral services, such as Lambda functions.
- It would be useful to have a standard set of observability practices that use existing plugins and libraries where possible (for instance, FastAPI has a Prometheus plugin).
- I'm assuming development in Python.

There are some standard/base application metrics that are useful for all applications in a technical context:

- Latency
- Memory usage
- Unique users
- Total users

We will then have application-specific aspects that record product-focused metrics. Examples, say for the Labs API project I am working on, include:

- Number of members using the deposit decorator
- Responsiveness of the proxy to the live REST API
- Number of attempted impolite requests
- Number of records registered in each batch

The specific product metrics that we collect should be discussed with product teams and other interested parties, potentially also including the user community.

Prometheus Metrics and their Challenges

There are several ways that you can use and provide Prometheus metrics.

For instance, where we are using ASGI/WSGI etc. servers in persistent contexts (e.g. running on EC2 instances), we could use default Prometheus metrics for our applications. Systems like FastAPI [have plugins that can handle the default/base instrumentation](#). These plugins will also create the API endpoint for scraping.

We could also implement custom metrics using Prometheus. Custom/product/application metrics can be handled either with a [custom Prometheus metric](#) or by pushing to AWS CloudWatch.

The problem though is that AWS Lambda functions (and other managed infrastructures) exist in an ephemeral state. While functions are “warm” a Prometheus metrics database will persist and can be scraped/queried. However, if the function is unused for between 5-15 minutes, the temporary container will shut down and the current metric data will be lost. Prometheus does not offer a reliable push method for storing metrics (despite the hope offered by the [Pushgateway](#) and [remote-write](#) options).

A Better Solution for AWS-Hosted Infrastructures: Structured Logging on CloudWatch

Use Built-In AWS CloudWatch Metrics/Logs

Many of the metrics that would be collected by Prometheus (e.g. latency, memory usage, CPU etc.) are logged by default by AWS Lambda in CloudWatch.

For instance, to monitor the latency of a Lambda function, first connect the AWS data source in Grafana. Then, from the dropdown on the dashboard query, ensure that “CloudWatch Logs” is selected, rather than CloudWatch Metrics. You can then use the query:

```
filter @type = "REPORT" |  
  stats avg(@duration), max(@duration), min(@duration) by bin(5m)
```

Likewise, memory use/consumption can be extracted using:

```
filter @type = "REPORT"  
  | stats max(@memorySize / 1000 / 1000) as provisionedMemoryMB,  
    min(@maxMemoryUsed / 1000 / 1000) as smallestMemoryRequestMB,  
    avg(@maxMemoryUsed / 1000 / 1000) as avgMemoryUsedMB,  
    max(@maxMemoryUsed / 1000 / 1000) as maxMemoryUsedMB,  
    provisionedMemoryMB - maxMemoryUsedMB as overProvisionedMB by bin(5m)
```

If using the default CloudWatch Metrics (rather than logs) then the API Gateway provides [a number of useful dimensions](#), including “count” for the number of hits.

These metrics should substitute for the defaults collected by frameworks like FastAPI.

Log Custom Metrics to CloudWatch

Boto3 and other client libraries for AWS provide the [ability to push metric data to CloudWatch](#), which we can then extract and visualize with Grafana.

To avoid repeated API pushes, it is advisable to build a list of collected metrics for each run and to push using a context manager's *exit* method (or other try/finally construct). Each run can push up to 1,000 custom metrics in a single call. See this [instrumentation.py](#) file for an example.

The “namespace” should be some kind of application name (e.g. “LabsAPIProxy”).

“Dimensions” in CloudWatch custom metrics are unique features by which you might want to aggregate or separate metrics (for instance: a route or HTTP status code).

These metrics can then be pulled into Grafana like any other.

Log in JSON Format

Logs written to CloudWatch should use [a JSON logger](#) so that these can be parsed automatically by Grafana and other querying tools.

Push Ephemeral Metrics to Storage/Logs (Not Recommended)

A potential solution if you really must use Prometheus metrics from a transient context is to scrape your own endpoint and then write an adapter to push these to CloudWatch or S3. This would, in general, be slow and delay the return of the Lambda function, increasing latency/response times.

Conclusion

AWS Lambda functions should use AWS CloudWatch for structured log storage and custom metric recording. This approach will work in all contexts but allows us to use a single query format to extract logging information. An example of how to implement this can be found in [the Labs API prototype](#).